QUESTION 17

Develop a Semantic Search System Using Cosine Similarity

Core Focus: Query Processing, Relevance Scoring and Ranked Retrieval

Technology: Python + Sentence Transformers + Scikit-learn

1. Aim

To develop a semantic search system that accepts a natural-language query, converts the query and stored documents into embeddings, calculates cosine similarity scores, and returns documents in order of semantic relevance.

2. Problem Statement

Traditional keyword search may fail when the query and the relevant document use different words for the same idea. A semantic search system addresses this problem by comparing vector representations of meaning. The system developed in this practical uses cosine similarity as the relevance function.

3. Objective

- Accept a natural-language search query.
- Represent the query in the same embedding space as the document collection.
- Measure the relevance of every document using cosine similarity.
- Rank search results from highest to lowest relevance.
- Return the highest-scoring document as the top search result.

4. Search System Architecture

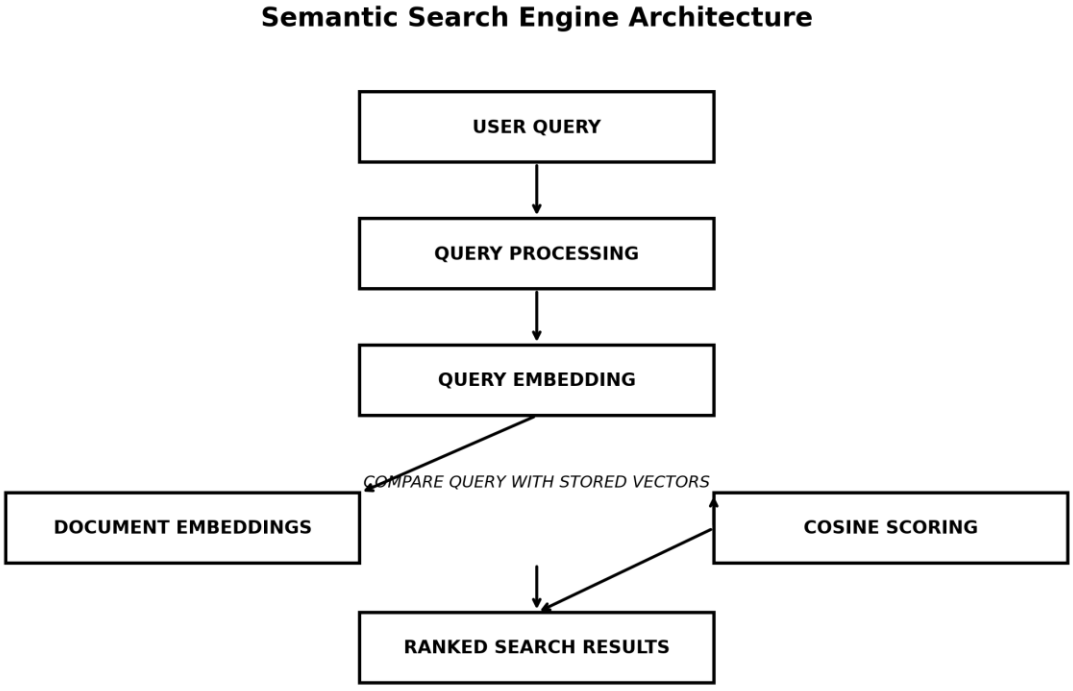

Figure 1: Query-centered architecture of the semantic search system.

5. Working Principle

The system treats semantic search as a retrieval and ranking problem. The document collection is first converted into vector representations. When a user submits a query, the query is encoded using the same embedding model. The system then compares the query vector with every stored document vector using cosine similarity. The resulting scores represent search relevance. Finally, the results are sorted in descending order.

6. Search Algorithm

1. Initialize the Sentence Transformer model.
2. Load the document collection.
3. Generate and retain an embedding for each document.
4. Receive the user's natural-language query.
5. Generate the query embedding.
6. Compare the query embedding against every document embedding.
7. Calculate one cosine similarity score for each document.
8. Sort the document-score pairs from highest to lowest.
9. Display the ranked search results and top match.

7. Relevance Calculation

Cosine similarity is used as the relevance score. It measures the directional similarity between the query vector and a document vector. A larger score means that the document is more closely aligned with the meaning represented by the query.

$$\textbf{Similarity} = (\textbf{Query} \cdot \textbf{Document}) / (||\textbf{Query}|| \times ||\textbf{Document}||)$$

8. Implementation Environment

Component	Used in Implementation
Language	Python
Embedding Model	all-MiniLM-L6-v2
Embedding Dimension	384
Similarity Method	Cosine Similarity
Libraries	sentence-transformers, scikit-learn
IDE	Visual Studio Code

9. Python Implementation

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "Machine learning enables computers to learn from historical data.",
    "Artificial intelligence allows machines to perform intelligent tasks.",
    "Deep learning uses neural networks to analyze complex data.",
    "Natural language processing helps computers understand human language.",
    "Computer networks allow devices to communicate and share information."
]

# Create searchable document vectors
document_embeddings = model.encode(documents)

# Receive and encode the search query
query = input("Enter your search query: ")
query_embedding = model.encode([query])

# Calculate relevance scores
similarity_scores = cosine_similarity(
    query_embedding,
    document_embeddings
)[0]

# Rank documents by relevance
ranked_results = sorted(
    zip(documents, similarity_scores),
    key=lambda item: item[1],
    reverse=True
)

# Display ranked search results
for rank, (document, score) in enumerate(ranked_results, 1):
    print(f"Rank {rank}")
    print(f"Similarity Score : {score:.4f}")
    print(f"Document         : {document}")

top_document, top_score = ranked_results[0]
print("Top Match:", top_document)
print("Score:", f"{top_score:.4f}")
```

10. Test Query

How can machines learn from previous information?

11. Actual Implementation Output

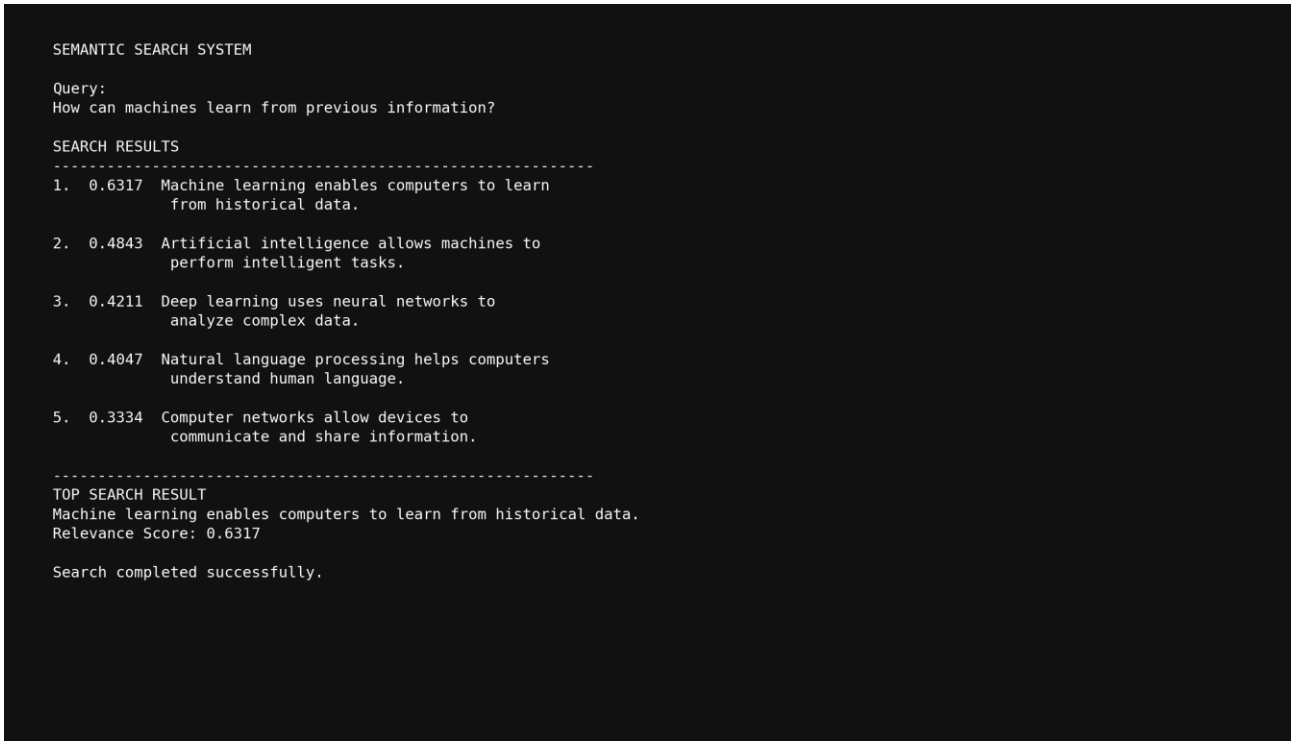

Figure 2: Actual ranked search output obtained from the implementation.

12. Search Results

Rank	Score	Relevance Level	Retrieved Document
1	0.6317	Highest	Machine learning enables computers to learn from historical data.
2	0.4843	High	Artificial intelligence allows machines to perform intelligent tasks.
3	0.4211	Moderate	Deep learning uses neural networks to analyze complex data.
4	0.4047	Moderate	Natural language processing helps computers understand human language.
5	0.3334	Lower	Computer networks

			allow devices to communicate and share information.

13. Search Ranking Analysis

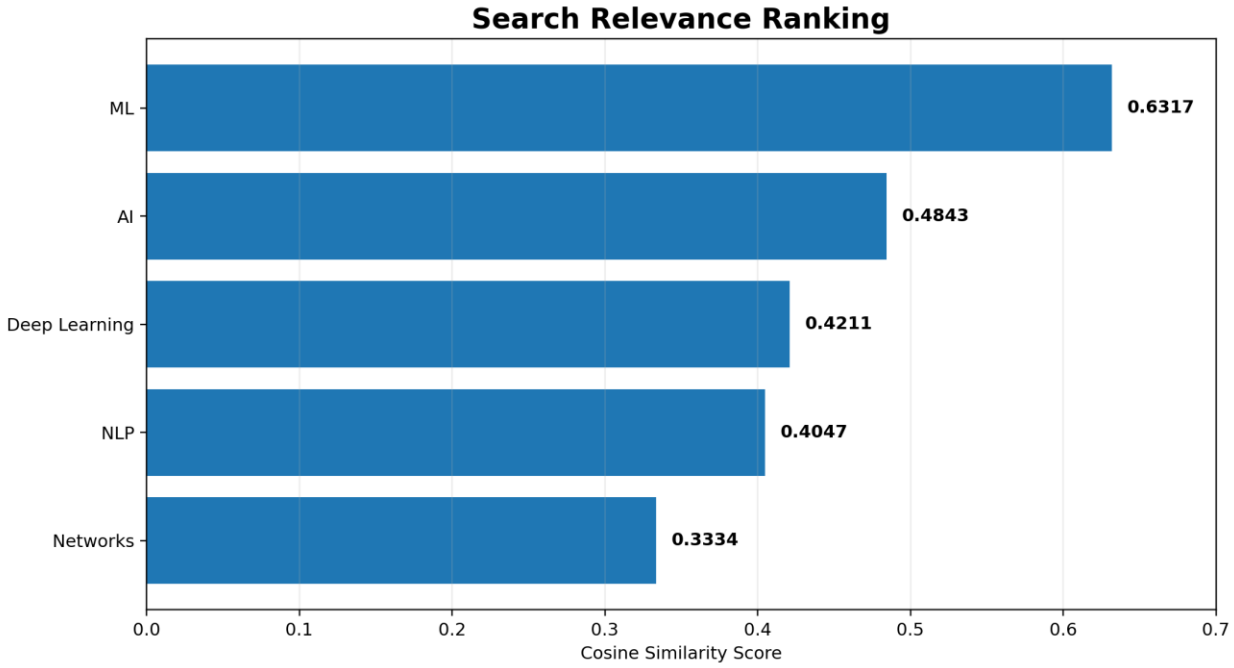

Figure 3: Ranked relevance scores produced by the semantic search engine.

The query is most closely related to learning from previous information. Therefore, the machine learning document receives the highest score of 0.6317. Artificial intelligence is the second-best result at 0.4843. Deep learning and natural language processing receive moderate scores, while computer networks receives the lowest score of 0.3334. This ranking demonstrates how cosine similarity can be used as a practical relevance function for semantic search.

14. Result

The semantic search system successfully processed the query and ranked all five documents. The top search result was “Machine learning enables computers to learn from historical data.” with a cosine similarity score of 0.6317.

15. Advantages

- Retrieves information according to semantic relevance.
- Reduces dependence on exact keyword matching.
- Provides an ordered list of search results.
- Simple to implement with modern embedding models.
- Can be extended to vector databases and large document collections.

16. Applications

- Semantic search engines
- Document retrieval systems
- Enterprise knowledge search
- AI-powered information retrieval
- Question-answering systems
- RAG retrieval modules

17. Conclusion

A semantic search system based on cosine similarity was successfully developed and tested. Unlike a simple embedding-generation demonstration, this implementation treats the embeddings as a searchable document collection and uses the query embedding to calculate relevance, rank results, and identify the top match. The system returned the machine learning document with a score of 0.6317, confirming successful semantic retrieval.